

SKIVE and gpt-oss: Architecture, Changes Made, and What Comes Next

A self-contained explanation of (1) the background concepts, (2) what is different about OpenAI's gpt-oss models, (3) how SKIVE is built and deployed, (4) what we changed to make SKIVE compatible with gpt-oss, and (5) what remains to be done. Architecture facts were confirmed directly in vLLM's source (`vllm/model_executor/models/gpt_oss.py`, `vllm/v1/attention/backends/flash_attn.py`, the KV-cache manager); model sizes come from the gpt-oss model card (arXiv 2508.10925).

1. Introduction

SKIVE is a **KV-cache eviction** system integrated into vLLM 0.23.0. While a language model generates text it keeps a cache of past tokens (the KV cache); for long outputs and many concurrent users this cache becomes the largest consumer of GPU memory and the reason a server stops admitting new requests. SKIVE keeps every request's cache under a fixed budget by discarding the least useful blocks of past tokens during generation, so more requests fit on the same GPU.

SKIVE was built and validated on models with one uniform kind of attention (Qwen2.5, Llama, DeepSeek-R1-Distill, Phi-3.5). gpt-oss-20b and gpt-oss-120b are built differently in three ways that each broke a hidden assumption. This document explains those differences, the fixes, and the remaining work. It does not present experimental results.

How to read it. Section 2 defines the handful of terms everything else depends on. Section 3 describes gpt-oss. Section 4 describes SKIVE's design and how it is deployed. Section 5 lists the changes, ordered from the smallest to the largest. Section 6 is the road map. A glossary and file map close the document.

2. Background concepts

2.1 The KV cache and why it grows

A transformer produces text one token at a time. To generate the next token it must attend to every previous token. Recomputing attention over the whole history each step would be prohibitively slow, so for every past token and every layer the model stores two vectors, a **key (K)** and a **value (V)**. This store is the **KV cache**. It grows with every token of every request:

```
KV bytes = 2 (K and V) x layers x kv_heads x head_dim x bytes_per_element x tokens
```

For gpt-oss-20b that is about 48 KB per token (in bf16), so a single 32k-token reasoning trace holds about 1.5 GB of cache, and a batch of many such requests fills a GPU quickly.

2.2 Prefill and decode

Generation has two phases. **Prefill** reads the prompt in one pass and fills the cache. **Decode** then produces tokens one by one, appending to the cache each step. SKIVE only ever removes cache entries during decode; the prompt is always processed in full.

2.3 Paged attention: blocks and block tables

vLLM does not store the cache contiguously. It divides GPU memory into fixed-size **blocks** (also called pages) of `block_size` tokens, 16 by default. Each request has a **block table**: the ordered list of physical block ids that hold its tokens, so logical position `i` lives in block `table[i // 16]`. The attention kernel reads through the table. Freeing memory therefore means returning whole blocks to a shared pool, which is why SKIVE evicts at block granularity.

2.4 KV-cache groups (the concept that matters most for gpt-oss)

Different attention types need different cache management. A **full-attention** layer must keep every past token. A **sliding-window** layer attends only to the last `W` tokens, so anything older can be discarded. vLLM handles this by putting each attention type into its own **KV-cache group**: a group has its own block size, its own per-request block table on the worker, and its own **manager** on the scheduler that allocates and frees blocks. A model with one attention type has one group; a model that mixes types has several, and a request then owns several block tables at once, one per group.

2.5 Attention sinks

In most transformers the first few tokens attract a disproportionate share of attention regardless of their content; they act as a "sink" for probability mass. Some models, gpt-oss among them, make this explicit: each attention head has a **learned sink logit** that enters the softmax alongside the real token logits, so a head can always route part of its attention "nowhere" instead of being forced to spread it over real tokens.

2.6 Grouped-query attention (GQA)

To shrink the cache, many models share one key/value head among several query heads. gpt-oss shares each KV head among 8 query heads (64 query heads, 8 KV heads). Consequences: the cache per token is 8x smaller than with one KV head per query head, and any per-head importance score must be combined across 8 query heads that may have very different scales.

3. What is different about gpt-oss

3.1 Architecture summary

property	gpt-oss-20b	gpt-oss-120b	confirmed in vLLM source
layers	24	36	model card
parameters, total / active per token	21B / 3.6B	117B / 5.1B	model card
feed-forward	Mixture of Experts: 32 experts, top-4	128 experts, top-4	<code>FusedMoE(num_experts=..., top_k=..., activation="swigluoai")</code>
attention heads	64 query / 8 KV (GQA x8), head_dim 64	same	<code>num_attention_heads, num_key_value_heads, head_dim</code>
attention pattern	alternating: even layers sliding-window 128 tokens, odd layers full attention	same	<code>sliding_window = config.sliding_window if layer_idx % 2 == 0 else None</code>
attention sinks	learned per-head sink logit in every layer	same	<code>self.sinks = nn.Parameter(...); Attention(..., sinks=self.sinks)</code>
positions	RoPE with YaRN scaling, 128k context	same	<code>rope_type: "yarn"</code>
expert weights	MXFP4 (about 13 GB)	MXFP4 (about 63 GB)	<code>_load_weights_mxfp4</code>

3.2 Comparison with the models SKIVE was validated on

property	Qwen2.5 / Llama / DeepSeek-Distill / Phi-3.5	gpt-oss	what it means for SKIVE
attention pattern	one type in every layer (full)	alternating sliding-window / full	vLLM creates two KV-cache groups ; a request has two block tables and two managers
attention sinks	none	learned per-head sink logit	the model's softmax has an extra denominator term
query/KV heads	GQA x4-x7, or MHA 32/32 (Phi)	GQA x8	per-head scores must be combined carefully
feed-forward	dense	MoE, MXFP4 weights	affects memory sizing only
attention backend	FlashAttention-2 works on any recent GPU	sinks are only supported by FlashAttention-3, which requires Hopper (compute capability 9.0)	on Ada GPUs (L4, L40S, RTX 4090) vLLM must use another backend for gpt-oss

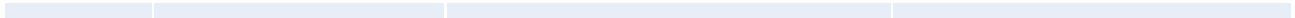
3.3 The three facts that break assumptions, explained

Two cache groups. With one attention type, "the block table" and "the manager" are unambiguous: index 0. With gpt-oss there are two of each, and the order is decided by vLLM at start-up, not fixed. Any code that hard-codes index 0 may operate on the sliding-window table, whose out-of-window blocks vLLM already replaces with null blocks by itself. Worse, each layer's cache tensor belongs to its own group, so block ids from one group's table are meaningless for a layer in the other group.

Sinks. The attention weight of token t in head h is $p_t = \exp(l_t) / (\exp(\text{sink}_h) + \sum_t \exp(l_t))$. Any importance score derived from a plain softmax (no sink term) is computed on a different distribution than the one the model actually uses.

GQA x8. A block's importance must be aggregated over 64 query heads whose score magnitudes differ widely; a plain sum lets a few heads dominate.

Backend. vLLM's FlashAttention backend refuses gpt-oss on GPUs below compute capability 9.0 ("sink not supported on compute capability < 9.0") and asserts "Sinks are only supported in FlashAttention 3". On an H100 gpt-oss runs through FlashAttention-3; on an L40S or L4 vLLM selects a different sink-capable backend. This matters because some of SKIVE's hooks live in the FlashAttention backend file.



4. SKIVE architecture and deployment

4.1 Design principles

1. **Hook-level integration, not a wrapper.** SKIVE manipulates vLLM's block tables and KV cache from inside the engine; a wrapper around the public API could not do this.
2. **Logic separated from integration.** All algorithms live in one package, `vllm/kv_evict/`; vLLM's own files receive only thin, flag-gated call-outs.
3. **Off by default, identical when off.** With `kv_evict_enabled=False` every hook is inert and vLLM behaves byte-for-byte like stock.
4. **No kernel changes.** SKIVE is pure Python; it never modifies CUDA code.
5. **Positions are never renumbered.** Evicted slots are repointed to a zeroed "null block"; every retained token keeps its position, so rotary embeddings stay exact.

4.2 The two parts of the code

The package `vllm/kv_evict/`:

module	responsibility
<code>scoring.py</code>	block-importance metrics
<code>selection.py</code>	which blocks are eligible, how many to evict, and the lowest-score pick (vectorized)
<code>compaction.py</code>	rewriting a block table to skip evicted blocks (sparse-gather)
<code>manager.py</code>	configuration object and budget bookkeeping
<code>integration.py</code>	the glue vLLM calls: group resolution, batched scoring, eviction, reclaim
<code>fused_attention.py</code>	optional Triton scoring kernel

The hooks in vLLM (13 small insertions in 6 files, all marked `SKIVE`):

where	what
<code>config/cache.py</code> , <code>engine/arg_utils.py</code>	the four flags: <code>kv_evict_enabled</code> , <code>kv_evict_budget</code> , <code>kv_evict_num_sink_blocks</code> , <code>kv_evict_num_local_blocks</code>
<code>v1/worker/gpu_model_runner.py</code>	after each decode step, call <code>skive_post_step</code> (wrapped in try/except so a failure degrades to full-cache decoding)
<code>v1/engine/core.py</code>	receive the list of freed blocks from the worker and release them in the scheduler's manager
<code>v1/attention/backends/flash_attn.py</code>	(a) capture the decode query for query-dependent metrics; (b) sparse-gather: drop evicted blocks from the kernel's read list
<code>v1/worker/gpu_model_runner.py</code>	raise loudly if eviction is enabled on the V2 runner (SKIVE targets the V1 runner)

4.3 The eviction pipeline, step by step

1. **Budget check.** After a decode step, find every request whose number of real (non-null) blocks

exceeds `kv_evict_budget`. This is a single vectorized operation over the batch.

2. **Protect the ends.** The first `kv_evict_num_sink_blocks` blocks (attention-sink tokens) and the last `kv_evict_num_local_blocks` blocks (recent context) are never candidates. Only the middle is scored.
3. **Score.** Compute an importance score per candidate block (Section 4.4).
4. **Select.** Evict the lowest-scoring candidates until the request is back at budget; ties resolve to the oldest block.
5. **Null-block substitution.** In the worker's block table, point each evicted slot at vLLM's zeroed null block. Length and positions are untouched.
6. **Reclaim.** The scheduler frees the physical block back to the pool. This is the step that lets vLLM admit more requests.
7. **Sparse-gather (FlashAttention backend only).** When building attention metadata, remove null slots from the kernel's gather list and shorten the effective length, so evicted blocks are neither read nor included in the softmax.

Cost controls: scores of static metrics are cached per block; all over-budget requests are scored in one GPU call with one GPU-to-CPU synchronization per step; eviction is deferred to every N steps unless a request is far over budget.

4.4 Scoring metrics

metric	formula per block	needs the decode query?	character
<code>vk_ratio</code> (default)	$\frac{ V_{\text{block}} _2}{(K_{\text{block}} _2 + \text{eps})}$, summed over layers	no	static; computed once per block and cached; works with any attention backend
<code>value_attention</code> (SKIVE)	$\sum_t \text{softmax}(q \cdot k_t) * v_t _1$	yes	attention weighted by value magnitude
<code>h2o</code>	$\sum_t \text{softmax}(q \cdot k_t)$	yes	summed attention mass
<code>snapkv</code>	$\max_t \text{softmax}(q \cdot k_t)$	yes	max-pooled attention
<code>streaming</code>	block position	no	keep sinks and recent only

The rationale for the default: the attention output is $\sum_t p_t v_t$ with $p_t \leq 1$, so a token's value magnitude bounds how much it can move the output; dividing by the key norm normalizes that against how strongly the block's keys can be selected. Because it needs no query and no softmax, `vk_ratio` costs nothing on the decode hot path and is independent of the attention kernel.

4.5 How SKIVE is deployed (this is the "gpt-oss deployment")

There are two equivalent ways to get SKIVE into a running vLLM, and understanding this removes most of the confusion.

Option A, the fork. The repository `Skive-V` is a complete copy of vLLM 0.23.0 with the 13 hooks already applied and `vllm/kv_evict/` vendored inside. Building it compiles vLLM's CUDA kernels (about an hour on a GPU box).

Option B, the overlay (what the gpt-oss scripts use). Because SKIVE is pure Python, the compiled kernels in the official vLLM 0.23.0 wheel can be reused unchanged:

1. `pip install vllm==0.23.0` (prebuilt wheel, about two minutes);
2. copy the 6 patched Python files and the `vllm/kv_evict/` package over the installed package (`overlay_skive.sh` does exactly this);
3. run any script, launching Python from outside the repository directory so the repository's own `vllm/` source tree does not shadow the installed package.

`run_gptoss.py` then loads gpt-oss with the team's inference settings (bf16, 32k context, `gpu_memory_utilization=0.90`, `enforce_eager`, temperature 0.6, top_p 0.95, `llm.chat`, `VLLM_USE_FLASHINFER_SAMPLER=0`) and switches SKIVE on with the gpt-oss configuration of Section 5.4. `deploy_and_test_gptoss.sh` chains all of this from a laptop to a remote GPU box over SSH: sync the working tree into the fork, copy the fork to the box, install and overlay, run the unit tests on the box, then run gpt-oss with SKIVE off and on.

Hardware note. On an H100 (compute capability 9.0) gpt-oss runs on FlashAttention-3. On every other GPU, including the RTX PRO 6000 Blackwell (12.0) and the L40S / L4 (8.9), vLLM serves gpt-oss's sinks through its Triton attention backend instead. SKIVE's eviction, reclaim and the `vk_ratio` metric are backend-independent; the two attention hooks (query capture and sparse-gather) are applied to both the FlashAttention and the Triton backend files, and the runners force whichever of the two the GPU supports, so the full SKIVE path is active on all of these cards. The 96 GB RTX PRO 6000 also holds gpt-oss-120b on a single GPU.

5. What we changed for gpt-oss compatibility

5.1 Changes ordered from minute to large

size	change	why
minute	block size read from the resolved group's block table	groups may use different block sizes
minute	query capture enabled for all query-dependent metrics	previously only <code>value_attention</code> captured the query
minute	gpt-oss configuration (Section 5.4), in particular <code>kv_evict_num_local_blocks >= 8</code>	8 blocks x 16 tokens = the 128-token sliding window is never evicted
small	sparse-gather gated to the full-attention group (<code>_skive_can_compact</code>)	the sliding-window group already holds vLLM's own null blocks; compacting them would shift the window

small	hybrid-block guard: eviction disabled with one warning if kernel and manager block sizes differ	the worker's block index would not match the manager's at reclaim
small	<code>vk_ratio</code> aggregates only the full-attention layers' caches, read through each layer's own <code>layer.kv_cache</code>	it used to walk every layer's cache with group-0 block ids
medium	sink-aware softmax (<code>_softmax_with_sink</code>) for every attention-based metric	matches gpt-oss's real attention distribution
medium	head aggregation option <code>SKIVE_HEAD_AGG = sum / max / zmax</code>	<code>zmax</code> (z-score per head, then max) keeps a token if any head needs it
medium	vectorized selection module <code>selection.py</code>	required by the updated integration; identical results
medium	reclaim routed by group: freed blocks carry <code>(request, index, group_id)</code>	blocks are released in the manager they belong to
large	multi-KV-group resolution (<code>_skive_full_group</code>): identify the true full-attention group, evict / score / reclaim only there, refuse to evict if it cannot be resolved	the structural fix; touches eviction, scoring and reclaim
large	fused Triton scoring kernel (<code>fused_attention.py</code>), made numerically stable and sink-aware	single-pass GPU scoring of the SKIVE metric, enabled by <code>SKIVE_SCORER=triton</code>

5.2 The large change in detail: multi-group resolution

At the first eviction the integration inspects vLLM's KV-cache configuration (falling back to the runner's attention groups) and classifies each group's spec: a group is "full attention" if its spec is not a sliding-window, chunked, Mamba, cross- or encoder-only spec and carries no window. It records that group's index and its layer names, and caches the result. From then on:

- the budget check and eviction use that group's block table;
- scoring iterates only that group's layers and reads each layer's own cache tensor, the one vLLM attached to the layer at start-up, instead of indexing a shared list by position;
- every freed block is tagged with the group id and released in that group's manager;
- if the model has several groups and none can be classified, SKIVE sets a flag, logs one line (`[SKIVE] ... unresolved`), and performs no eviction at all. Refusing is deliberate: guessing group 0 on gpt-oss could mean evicting from the sliding-window table.

5.3 The medium changes in detail

Sink-aware softmax. For a layer with sinks, the sink logit is appended to the token logits before the softmax and dropped afterwards, which yields exactly $\exp(l_t) / (\exp(\text{sink}) + \sum \exp(l))$. Within a head the ordering of tokens is unchanged, so eviction decisions based on a single head are unaffected; across heads the weighting now matches the model.

Head aggregation. With `zmax`, each head's token scores are standardized over that head's tokens (z-score) so heads with large raw magnitudes no longer dominate; the per-token score is then the maximum across heads. This follows the GQA treatment in TriAttention (arXiv 2604.04921, Section 4.3).

Group-routed reclaim. The worker previously reported `(request, index)` pairs and the scheduler assumed manager 0. Pairs now carry the group id; legacy pairs still map to group 0 for compatibility.

5.4 Resulting gpt-oss configuration

setting	value	reason
<code>kv_evict_num_local_blocks</code>	8 or more	keeps the full 128-token sliding window
<code>kv_evict_budget</code>	128 blocks (2048 tokens)	TriAttention's reasoning budget
<code>kv_evict_num_sink_blocks</code>	2	attention-sink tokens
<code>SKIVE_METRIC</code>	<code>vk_ratio</code> (backend-independent) or <code>value_attention</code>	see Section 4.4
<code>SKIVE_HEAD_AGG</code>	<code>zmax</code>	GQA-aware aggregation (query-dependent metrics)
<code>SKIVE_SCORE_LAYER_STRIDE</code> / <code>SKIVE_EVICT_EVERY</code>	4 / 16	cheaper scoring; deferred eviction
<code>enable_prefix_caching</code>	False	reclaim assumes no shared-block reference counts
<code>VLLM_USE_V2_MODEL_RUNNER</code>	0	SKIVE's hooks live in the V1 runner

5.5 Tooling added for gpt-oss

`run_gptoss.py` (the team's recipe with SKIVE on/off), `run_aime24_gptoss.py` (the TriAttention AIME-2024 protocol), `overlay_skive.sh` (prebuilt wheel + overlay), `deploy_and_test_gptoss.sh` (laptop-to-box deployment), `finish_sync.sh` (working tree to fork, gate, compile, tests), and unit tests for the group resolution, group-routed reclaim, guards, sink-aware softmax, `zmax`, and the fused scorer.

6. What needs to be done next

Ordered by importance.

1. **End-to-end validation on gpt-oss.** The multi-group integration was implemented against vLLM's group / coordinator / block-table code and is covered by unit tests, but the first full gpt-oss run is the integration test. It must confirm three things from the logs: the full-attention group is resolved (group id in the debug line), evictions and reclaims fire, and outputs remain coherent. An H100 exercises the complete FlashAttention-3 path; an L40S/L4 exercises the backend-independent path only.
2. **Task-level evaluation at the target budget.** Run the AIME-2024 protocol (30 problems, 8 samples, 2048-token budget) for gpt-oss-20b with SKIVE off and on, then gpt-oss-120b with tensor parallelism. This is what turns the design into evidence.
3. **Choice of metric on gpt-oss.** `vk_ratio` is backend-independent and free but static; the query-dependent metrics are more selective but require the FlashAttention query hook (H100). A hybrid that uses the query-dependent score where available and falls back to `vk_ratio` elsewhere is a natural next step.
4. **Backend coverage.** The two attention hooks now exist in both the FlashAttention and the Triton backend, which covers Hopper, the RTX PRO 6000 and Ada cards. FlashInfer (the default on B200-class SM100 GPUs) still lacks them; on such a GPU the runner must force the Triton backend, at some attention-speed cost, until the hooks are ported there too.
5. **Finish the fused scoring kernel.** The stable, sink-aware Triton scorer is implemented and unit-tested against the torch path; it should be profiled against the batched torch implementation and enabled by default only if it wins.
6. **Asynchronous scoring.** Overlap block scoring with the weight-streaming phase of decode on a side CUDA stream, so eviction overhead approaches zero even in compute-bound regimes.
7. **Prefix caching.** Reclaim currently assumes prefix caching is off (no shared-block reference counts). Supporting it requires reference-count-aware freeing.
8. **Per-layer budgets.** vLLM's block table is shared by all layers of a group, so a block is evicted from every layer of that group at once; methods that assign different budgets per layer (PyramidKV-style) cannot be expressed. This is an architectural limitation to document, or a larger change to vLLM's cache layout.
9. **CUDA graphs.** The scripts run with `enforce_eager=True`; making eviction compatible with CUDA-graph capture (compaction outside the captured region) would recover the graph speed-up.
10. **Upstreaming.** The six hook sites are small; proposing them to vLLM would remove the version pin to 0.23.0 and let SKIVE track vLLM releases.

7. Glossary

- **KV cache** - stored key/value vectors of past tokens; grows with every token and request.
- **Block / page** - a fixed-size chunk of the KV cache (16 tokens by default).
- **Block table** - a request's ordered list of physical block ids, one entry per logical block.
- **KV-cache group** - vLLM's unit of cache management for one attention type; has its own block tables and manager.
- **Null block** - vLLM's reserved zeroed block (id 0); evicted slots point at it.
- **Sink / local blocks** - the first / last blocks of a request, never evicted.

- **Budget** - the maximum number of real blocks a request may keep.
- **Attention sink (gpt-oss)** - a learned per-head logit that joins the softmax denominator.
- **GQA** - grouped-query attention; several query heads share one KV head.
- **Sparse-gather** - removing evicted blocks from the attention kernel's read list.
- **Overlay** - installing SKIVE's Python files over the official vLLM wheel instead of building the fork.

8. File map

path	purpose
<code>kv_evict/integration.py</code>	group resolution, batched scoring, eviction, reclaim (start here)
<code>kv_evict/scoring.py</code> , <code>selection.py</code> , <code>compaction.py</code> , <code>manager.py</code>	metrics, selection, sparse-gather compaction, configuration
<code>kv_evict/fused_attention.py</code>	Triton scoring kernel (optional)
<code>stage4/patch_vllm.py</code> , <code>skive_patches/</code>	the 13 reproducible hooks as a patcher and as diffs
<code>stage6/run_gptoss.py</code> , <code>run_aime24_gptoss.py</code>	gpt-oss runners
<code>stage6/overlay_skive.sh</code> , <code>deploy_and_test_gptoss.sh</code> , <code>finish_sync.sh</code>	deployment tooling
<code>tests/test_multigroup.py</code> , <code>test_selection.py</code> , <code>test_gptoss_scoring.py</code>	unit tests for the gpt-oss changes
<code>GPTOSS.md</code> , <code>UPGRADE_NOTES.md</code>	operating guide and change log